Grouped-Query Attention and Partial RoPE Under a Fixed FLOP Budget: A Controlled Study of a Small Language Model Trained from Scratch

Jerry Yang Shao Ern (A0281549U)

Repository: `https://github.com/mcxraider/modded-nanochat` (experiment namespace `d8-v5`; code commit **[commit hash of the d8-v5 launch]**).
Group members: Jerry Yang Shao Ern, student number A0281549U.

Compute budget (declared before any training run). Hardware: one NVIDIA H100 SXM 80 GB on Modal, BF16 autocast, FlashAttention-3. Per-run allowance: at most $C = 1.385 \times 10^{17}$ *estimated* training FLOPs, the cost of the baseline B0 trained on upstream nanochat's compute-optimal horizon at depth 8 (1,680 updates $\times$ 262,144 tokens $\times$ 314,575,872 FLOPs/token). FLOPs are counted with nanochat's estimator (6 FLOPs per matmul parameter per token plus windowed attention), so each architecture receives the largest whole number of updates whose estimate fits in C. A 2-hour wall-clock cap applies to every run as a safety stop only; no comparable run reached it (each needed about **[6–9]** minutes). Pilots, numerical preflight checks and tokenizer training are excluded from the budget and reported separately.

Initialisation. Every model is trained from random initialisation. No pretrained weights, embeddings or other pretrained neural components are used. We reuse the nanochat architecture and code, its BPE tokenizer recipe (trained by us on the training shards, vocabulary 32,768) and the ClimbMix corpus.

Evaluation objective. Bits per byte (BPB) of next-token prediction on held-out ClimbMix web text: 2^{24} tokens from shard 06542 (validation, used for curves and any decision) and, separately, 2^{24} tokens from shard 06541 (test, evaluated once per run after training; the primary metric). BPB normalises the summed cross-entropy by the UTF-8 byte count of the targets, so it is independent of tokenisation and measures the quantity the budget is spent on, namely language modelling of the training distribution. It does not measure downstream reasoning, instruction following or long-context use, and inference speed and memory are reported as separate metrics.

1 Introduction and research question

Two attention changes appear in almost every recent open model: *grouped-query attention* (GQA) [1, 15], which lets several query heads share one key/value (K/V) head, and *partial rotary embeddings* [5, 17], which rotate only a fraction of each head's dimensions. Both are usually justified by inference cost (a smaller K/V cache) or by evidence from large models, not by what they do to a small model trained from scratch on a tight compute budget. The brief asks how to obtain the best model within a fixed per-run allowance, which makes this the relevant question. GQA removes training FLOPs, so at fixed compute a GQA model trains on *more* tokens than its multi-head (MHA) counterpart. Whether it ends up better or worse therefore depends on two competing effects: a per-token quality cost from sharing K/V heads, and a gain from the extra data. Our research question is:

> *At depth 8 and a fixed per-run budget of C estimated training FLOPs, what do GQA and partial RoPE cost or buy in held-out BPB, training throughput and inference memory/speed, and how much of GQA's fixed-compute result comes from the extra tokens its FLOP saving buys?*

We wrote the hypotheses, resolution thresholds and analysis rules in `experiments/experiments.md` before running any training job. Every outcome below is reported against its prediction, including the null and negative ones. Our contributions are: (i) a fixed-compute comparison of MHA, GQA-2 and MQA with a seed-noise estimate on *both* the baseline and the main variant; (ii) a dose–response over the rotary fraction; (iii) two controls for confounds that the naive comparison hides: the *value embeddings* in this architecture shrink with the K/V head count (a 2×2 design), and GQA's per-token penalty is mixed with its extra data (a matched-token ablation); and (iv) an inference benchmark that tests where the K/V saving actually appears on an H100.

2 Background

Multi-query and grouped-query attention. In MHA [18] every query head h has its own key and value projections. During autoregressive decoding, each step must read the whole cache of past keys and values, so at large batch or long context decoding is limited by memory bandwidth rather than arithmetic [14, 15]. MQA [15] shares a single K/V head across all query heads and cuts the cache by a factor of n_{head}. GQA [1] interpolates between the two with n_{kv} groups, and reports quality close to MHA when large checkpoints are *uptrained* into GQA form. That evidence comes from converted T5-XXL models, not from small models trained from scratch, which is why we test the claim directly. In training, GQA saves only the K/V projection matmuls: the attention score and value products still run over all n_{head} query heads. At our size this is 8% of B0's estimated FLOPs for full K/V, so GQA-2 saves 4.2% and MQA 6.4% (Table 1).

Rotary embeddings and partial RoPE. RoPE [17] rotates pairs of query/key dimensions by angles proportional to position, so the dot product depends on relative offset. Each of the $d/2$ pairs has frequency $\theta_i = 10000^{-2i/d}$. GPT-NeoX [5] rotates only 25% of each head and leaves the rest position-free. Barbero et al. [3] found that a trained Gemma model uses the high-frequency pairs for positional patterns and the lowest frequencies as almost position-independent "semantic" channels, and that removing the lowest frequencies can help. Partial RoPE achieves something similar by construction. A detail matters for interpreting our dose–response: in our implementation the frequencies are recomputed for the rotated prefix (base 10^4 over d_{rot} dimensions), so R25 keeps the same highest and lowest frequency as full RoPE but samples the spectrum $4\times$ more coarsely (16 pairs instead of 64), and adds 96 dimensions with no positional signal at all.

Value embeddings. nanochat's attention adds a learned, token-indexed *value embedding* to the values in alternating layers, scaled by an input-dependent gate. This is a form of ResFormer value-residual learning [19], which reports the same loss with 16% fewer parameters. The embedding width equals the value width, $n_{\text{kv}} \cdot d_{\text{head}}$. At depth 8 these tables hold 67.1M of B0's 125.8M parameters, so halving the K/V heads removes 33.5M embedding parameters, about 16 times the 2.1M removed from the K/V projections. Nanochat's FLOP estimator treats embedding lookups as free, so this change is invisible to the budget. A naive "GQA vs MHA" comparison in this codebase is therefore also a "half vs full value embeddings" comparison, and Experiment E5 separates the two.

Compute-optimal training. Chinchilla [9] models loss as $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ and finds roughly 20 tokens per parameter compute-optimal. Kaplan et al. [11] first pointed out that embedding parameters behave differently in such fits. nanochat [12] fits its own ratio of 10.5 tokens per *scaling* parameter (transformer matrices plus unembedding) and we adopt its horizon for B0. We use the Chinchilla data term only to make an order-of-magnitude prediction of what GQA's extra 4–6% of tokens should be worth (§3.3).

3 Methods and experimental setup

3.1 Model, data and training procedure

Baseline B0. B0 is the nanochat ClimbMix leaderboard model at upstream commit `324e69c` (the configuration immediately before the "autoresearch" rounds), rescaled to depth 8: 8 layers, width 512, 4 heads of dimension 128, context 2,048, vocabulary 32,768. It keeps that commit's design: pre-norm with parameter-free RMSNorm, QK-norm [8], squared-ReLU MLP [16] of width $4\times$, untied embeddings, logit soft-capping, learnable residual and input-skip scalars, gated value embeddings in alternating layers, and sliding-window attention with the pattern SSSL (short windows of 1,024 tokens, every fourth and the final layer full) [4]. We made one deliberate change: we train in BF16 rather than the leaderboard's FP8, so that numerical precision cannot interact with the head shapes.

Variants. We extended the model with three switches, each verified by CPU unit tests and a GPU preflight (§3.5): the number of K/V heads $n_{\text{kv}} \in \{4, 2, 1\}$; the rotary fraction $f \in \{1, 0.75, 0.25\}$, where the first $f \cdot d_{\text{head}}$ dimensions of q and k are rotated and the rest pass through unchanged; and the MLP ratio and value-embedding on/off flags used by the controls. Table 1 lists the eight resulting architectures.

Data and tokenizer. We use NVIDIA ClimbMix [7], a 400B-token web mixture produced by clustering-based mixture search, in the shard format used by nanochat. Upstream switched to ClimbMix because it gave

Table 1: The eight architectures (depth 8, width 512, 4 query heads of dimension 128). FLOPs/token is nanochat's estimate. "Updates" is the largest whole number of 262,144-token updates within C. Scaling parameters are the transformer matrices plus the unembedding, and total parameters add the token and value embeddings. KV cache is in bytes per token (BF16, all layers).

Name	Change from B0	FLOPs/token	Updates	Tokens	Scaling / total params	KV B/token
B0	— (MHA, $n_{\text{kv}} = 4$)	314.58M	1,680	0.440B	41.94M / 125.83M	16,384
GQA-2	$n_{\text{kv}} = 2$	301.99M	1,750	0.459B	39.85M / 90.18M	8,192
GQA-2-R75	GQA-2, 75% RoPE	301.99M	1,750	0.459B	39.85M / 90.18M	8,192
GQA-2-R25	GQA-2, 25% RoPE	301.99M	1,750	0.459B	39.85M / 90.18M	8,192
GQA-2-PM	GQA-2, MLP 4.5×	314.57M	1,680	0.440B	41.94M / 92.27M	8,192
MQA (GQA-4)	$n_{\text{kv}} = 1$	295.70M	1,787	0.468B	38.80M / 72.35M	4,096
B0-noVE	B0, no value emb.	314.57M	1,680	0.440B	41.94M / 58.72M	16,384
GQA-2-noVE	GQA-2, no value emb.	301.99M	1,750	0.459B	39.85M / 56.62M	8,192

clear gains over FineWeb-Edu and other corpora on its leaderboard. The training pool is shards 0–47. Shard 06542 (the last, which nanochat treats as validation) is our validation split, and shard 06541 is a held-out test split that is never used for any decision. We trained the nanochat BPE tokenizer (vocabulary 2^{15}) ourselves. Its trainer reads files in index order and stops after 2×10^9 characters, so it sees only the first training shards and never the test shard **[confirm from the `tok_train` log]**. Documents are packed into 2,048-token rows with nanochat's BOS-aligned best-fit packer. The longest run consumes **[x.xx]**% of the pool, so no run repeats data (`data_epoch` = 1 is checked).

Optimisation. Following the pinned upstream training script, matrices are updated with Muon [10] (Polar-Express orthogonalisation [2] and NorMuon-style factored second moments [13]) and the embeddings, unembedding and scalars with AdamW. Upstream's own depth-8 rules set the global batch to 262,144 tokens, all learning rates scaled by $\sqrt{262144/524288} = 0.707$, and the Muon weight decay to 0.371, computed once from B0 and *frozen for every variant*. The schedule has no warm-up, a constant learning rate for 50% of updates and then linear decay to zero. Weight decay decays linearly with progress, and Muon momentum warms up from 0.85 to 0.95 over 300 updates. We tuned no hyperparameters for any variant (§5).

3.2 The FLOP budget and why we chose it

We chose a FLOP budget rather than wall-clock time because (a) it is hardware-independent and reproducible, while throughput on a rented GPU varies with the node; (b) it is the quantity scaling laws reason about; and (c) it rewards an architecture exactly for the arithmetic it removes, which is the claimed advantage of GQA in training. The cost is that the estimator ignores work that is not a matmul, so it understates the true cost of value embeddings and treats partial RoPE as free. We report measured throughput next to every comparison so that readers can see how the ranking would change under a time budget (§4.4). Training stops at the token target and refuses to start if the target exceeds C. All fixed-compute runs use at least 99.98% of C.

3.3 Experimental design

We ran 14 training runs. Apart from the stated change, all share data, tokenizer, context, batch, optimiser, schedule and evaluation text, and all variants use seed 1 unless noted. The seed sets both the initialisation and the training-shard rotation, so seed noise includes data-order noise.

- **E1, noise floor** (`d8-b0-s1..s3`): B0 with seeds 1–3. The sample s.d. s of test BPB defines the resolution threshold.
- **E2, main comparison** (`d8-gqa2-s1..s3`, `-r75`): GQA-2 (seeds 1–3) against the B0 mean, and GQA-2-R75 against both. The three GQA-2 seeds also check that the noise is similar across architectures.
- **E3, parameter-matched control** (`d8-gqa2-pm`): GQA-2-PM spends GQA's saving on a wider MLP (2,304 instead of 2,048) instead of extra tokens. It matches B0's scaling parameters to 256 parameters and its FLOPs to 0.0005%.
- **E4, dose–response** (`d8-gqa4`, `d8-gqa2-r25`): MQA (K/V axis, against B0 and GQA-2) and GQA-2-R25 (rotary axis, against GQA-2 and R75).
- **E5, value-embedding** 2×2 (`d8-b0-nove`, `d8-gqa2-nove`): B0-noVE and GQA-2-noVE. With B0 and GQA-2 these cross $\{4, 2\}$ K/V heads with value embeddings {on, off}.
- **E6, matched-token ablation** (`d8-gqa2-tok`, `d8-gqa4-tok`): GQA-2 and MQA trained on B0's 1,680 updates (96% and 94% of C). The difference to their fixed-compute twin (same seed, so the same initial

weights and batches up to update 1,680) is the value of the extra tokens, and the difference to B0 is the per-token cost of K/V sharing.

Pre-registered predictions. We wrote these predictions down before any run. **H2a:** GQA-2 is not resolved-worse than B0 at fixed compute. **H2b:** GQA-2's training-throughput gain is below 5%. **H2c:** R75 is not resolved from full RoPE in BPB and differs by under 2% in throughput. **H3:** PM is within the threshold of B0. **H4a:** BPB degrades monotonically with K/V sharing, and MQA is resolved-worse than B0. **H4b:** R25 is not resolved from R75 or full RoPE. **H5a:** removing value embeddings costs B0 a resolved loss. **H5b/c:** GQA's gap is smaller without value embeddings (a positive interaction, probably unresolved). **H6:** from the Chinchilla data term ($B = 410.7$, $\beta = 0.28$, $B/D^{\beta} \approx 1.56$ nats at $D = 0.44$B), 4.2% / 6.4% more tokens should be worth about 0.018/0.027 nats, i.e. roughly 0.005–0.006 / 0.008–0.009 BPB at 4.2–4.8 bytes per token. **HIa–c (inference):** decode never materialises expanded K/V; at batch 1 the variants decode within 5% of each other; at batch 64, GQA's decode advantage grows with prompt length up to the training context and then plateaus.

Statistical procedure. A difference counts as *resolved* only if it exceeds $2\sqrt{2}\,s$, a two-sigma bound on the difference of two single runs, where s is the s.d. of the three B0 seeds. This rule is crude: s has two degrees of freedom. By simulation it flags about 18% of null single-vs-single contrasts, so across our ≈ 17 contrasts two or three spurious "resolved" flags are expected [6]. We therefore pre-registered three secondary analyses and report where they disagree with the primary rule: (i) a bound that accounts for seed means, $2s\sqrt{1/n_1 + 1/n_2}$; (ii) a Student-t 95% interval using s_p, the s.d. pooled over the B0 and GQA-2 groups (df = 4); and (iii) a *paired* analysis that uses the shared data order of same-seed runs (E2 seed pairs and E6 twins). Validation curves are compared only between runs with the same token target, because a longer run's learning-rate decay starts later.

3.4 Inference benchmark

After training, each final checkpoint is benchmarked on the H100 with fixed random prompts, greedy decoding of exactly 256 tokens, a batch size $\in \{1, 4, 16, 64\}$ and a prompt length $\in \{128, 1792, 8192\}$. A 1,792-token prompt plus 256 generated tokens exactly fills the 2,048 training context. The 8,192-token prompt exceeds the context, so we use it only to measure speed and memory, with no quality claim. We report prefill throughput, time to first token, decode throughput (the remaining 255 tokens), peak memory and the allocated KV cache, and compare them with the analytic cache size and the K/V bytes read per decoded token. Execution is eager. Weights are cast to BF16 once, and the cache position is kept on the host to avoid a GPU synchronisation per step.

3.5 Validity checks before the main runs

CPU unit tests check that full RoPE reproduces the historical code exactly, that the un-rotated dimensions pass through unchanged, that GQA projection and cache shapes are correct, that cached decoding matches full-sequence decoding, and that resuming from a checkpoint is bit-identical. On the H100, a preflight for all eight architectures compares FA3 with PyTorch SDPA (logits, loss and gradients) and the compiled model with eager execution (per-tensor relative gradient error $\leq 5\%$) across the sliding-window boundary. The largest observed error was **[x.x%]** on **[parameter]**. A 10-update pilot per architecture then confirmed FA3 selection, no out-of-memory error anywhere in the inference grid, a decode working set below half the cache (so K/V heads are not expanded), and data-loader wait time below 2% of each update, so measured throughput reflects the GPU rather than the CPU packer. All pilots passed and no setting was changed afterwards (`results/pilot_d8-v5.md`). The suite cost **[$xx.xx]** against a projected $21.38.

4 Results

4.1 Noise floor (E1)

The three B0 seeds reach test BPB **[a]**, **[b]** and **[c]** (mean **[m]**), so $s =$ **[s]** and the resolution threshold is $2\sqrt{2}\,s =$ **[t]** BPB. The three GQA-2 seeds give $s_{\text{GQA-2}} =$ **[s2]** and the pooled $s_p =$ **[sp]** (df = 4). The prediction $s \in [0.001, 0.003]$ **[held / failed]**. **[Comment on whether the two groups' noise is similar, which the threshold assumes. Comment on validation–test agreement: is the gap between validation and test BPB constant across runs, i.e. is the test shard simply harder/easier?]**

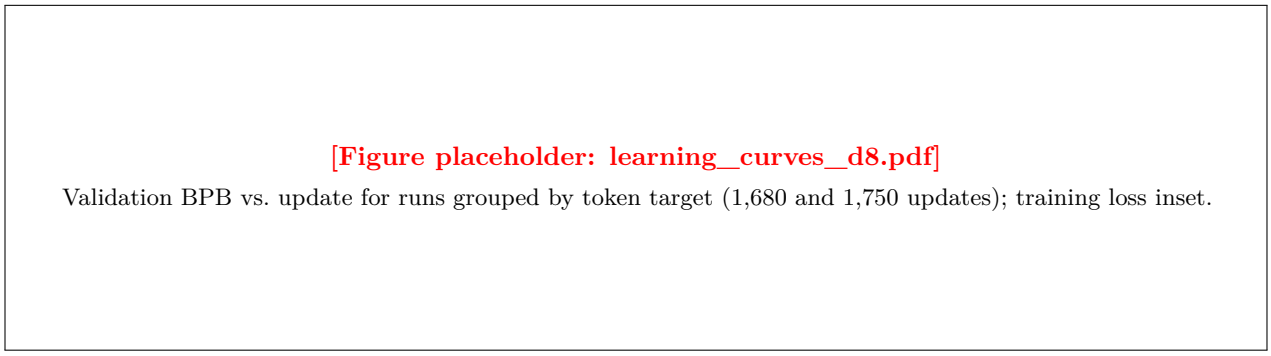


Figure 1: Learning curves. Curves are compared only within the same token target.

Table 2: Final results. Δ is test BPB minus the primary reference (negative is better). "Res." is the primary rule $|\Delta| > 2\sqrt{2}s$, "means" the seed-mean rule, and "t" the pooled-t 95% interval. Throughput is the median steady-state training tokens/s. Seed groups are means of three runs.

Run	Test BPB	Val BPB	Reference	Δ test	Res.	means	t	Tok/s (rel. B0)
B0 (3 seeds)	[]	[]	—	—	—	—	—	1.000
GQA-2 (3 seeds)	[]	[]	B0	[]	[]	[]	[]	[]
GQA-2-R75	[]	[]	GQA-2	[]	[]	[]	[]	[]
GQA-2-R25	[]	[]	GQA-2	[]	[]	[]	[]	[]
GQA-2-PM	[]	[]	B0	[]	[]	[]	[]	[]
MQA	[]	[]	B0	[]	[]	[]	[]	[]
B0-noVE	[]	[]	B0	[]	[]	[]	[]	[]
GQA-2-noVE	[]	[]	B0-noVE	[]	[]	[]	[]	[]
GQA-2-tok	[]	[]	B0	[]	[]	[]	[]	[]
MQA-tok	[]	[]	B0	[]	[]	[]	[]	[]

4.2 Quality at fixed compute (E2–E5)

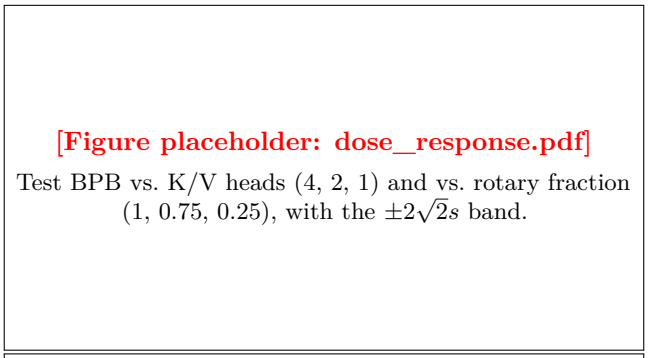
GQA-2 vs. MHA (H2a). At fixed compute, GQA-2 reaches [x] test BPB against B0's [y] (Δ = [d], [resolved / unresolved]). The paired analysis over the three same-seed pairs gives a mean difference of [d] with 95% interval [[l,u]], and $s_d/(\sqrt{2}s)$ = [r], so the shared data order removes [%] of the seed noise. **[Interpretation: does H2a hold? Note that this contrast changes K/V heads, value-embedding size and token count at once; E5 and E6 below decompose it.]**

K/V dose–response (H4a). **[BPB at $n_{\mathrm{kv}} = 4, 2, 1$; monotone or not? MQA vs B0 resolved? Recall that MQA has 6.4% more tokens and a quarter of B0's value-embedding parameters, and that its value gate is a 1×32 matrix, for which Muon's orthogonalised update reduces to a normalised vector.]**

Partial RoPE (H2c, H4b). **[R75 and R25 against GQA-2, all at 1,750 updates, so curves are directly comparable (Fig. 2). Interpret with the frequency argument from §2: R25 keeps the extreme frequencies but drops 75% of the rotated channels in favour of position-free ones.]**

Value embeddings (E5, H5a–c). **[Is the value-embedding effect resolved (H5a)? Is the GQA gap smaller without value embeddings (H5b)? How large is the interaction relative to its bound? If the GQA penalty largely disappears without value embeddings, then the "GQA cost" in this architecture is mostly a value-embedding capacity cost, which matters for anyone porting GQA into a model with value residuals.]**

Spending the saving on parameters instead of tokens (E3, H3). **[GQA-2-PM vs B0 and vs GQA-2. PM and GQA-2 are two ways to spend the same FLOP saving: on 256 extra MLP units or on 70 extra updates. Which wins? Remember that PM still has 33.5M fewer embedding parameters than B0, and that Muon gives `c_fc` about 6% more learning rate in the wider MLP.]**



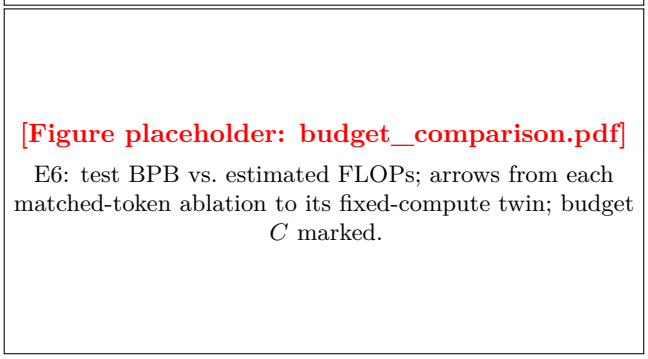


Figure 2: Left: dose–response along the K/V and rotary axes. Right: what the extra tokens buy (E6).

Table 3: E5 2×2 on test BPB. The interaction is (GQA-2 − B0) − (GQA-2-noVE − B0-noVE), with two-sigma bound $2s\sqrt{1/3 + 1/3 + 1 + 1} \approx 3.3s$.

	Value emb. on	Value emb. off	Effect of removing
MHA ($n_{\text{kv}} = 4$)	[] (B0, $n = 3$)	[]	[]
GQA-2 ($n_{\text{kv}} = 2$)	[] ($n = 3$)	[]	[]
Effect of GQA-2	[]	[]	interaction [] (bound [])

4.3 What the extra tokens buy (E6, H6)

[GQA-2(s1) − GQA-2-tok = Δ_1 and MQA − MQA-tok = Δ_2 against the predicted -0.005 and -0.009 BPB. GQA-2-tok − B0 and MQA-tok − B0 are single-seed estimates of the per-token cost of K/V sharing. Decompose the E2 result: fixed-compute gap = per-token cost + data gain. State the paired-noise assumption ($2s_d$ threshold) and whether the Chinchilla-based prediction was of the right order.]

4.4 Training throughput and memory

The pilots (8 timed updates each) and full runs agree that GQA buys little training speed at this size (Table 4). GQA-2 removes 4.2% of estimated FLOPs but is only **[1.6]**% faster, and MQA removes 6.4% and is **[5.5]**% faster. Throughput in FLOP/s falls slightly as K/V heads are removed (379 to 369 TFLOP/s for GQA-2 in the pilot), which is consistent with the narrower 512×256 K/V matmuls using the tensor cores less efficiently than the square ones and with attention, which GQA does not shrink, taking a larger share of the step. Two observations show the limits of the FLOP budget. First, removing the value embeddings leaves the FLOP estimate unchanged but makes B0 **[2.0]**% faster: the lookups, gates and their AdamW state cost real time that the estimator does not count. Second, GQA-2-PM is FLOP-matched to B0 but **[1.5]**% slower, because its 2,304-wide MLP is less efficient on the hardware. Under a wall-clock budget, both effects would shift tokens towards the variants without value embeddings and away from PM. Partial RoPE changes throughput by under 1% (H2c). **[Peak training VRAM per architecture; evaluation peak separately.]**

4.5 Inference: where the K/V saving appears

We expected the K/V saving to matter only when K/V reads dominate the bytes moved per decode step. Analytically, at batch 1 and a 128-token prompt, the 84 MB of BF16 matmul weights dominate, and K/V

Table 4: Training throughput (full-run steady-state median; pilot value in parentheses) and achieved TFLOP/s (= tokens/s × FLOPs/token; H100 dense BF16 peak 989). **[Replace pilot-only entries with full-run values.]**

Architecture	Tokens/s (M)	vs. B0	FLOP saving	Achieved TFLOP/s (MFU)
B0	[] (1.205)	1.000	—	[] (379, 38%)
GQA-2	[] (1.223)	[] (1.016)	4.2%	[] (369, 37%)
GQA-2-R75	[] (1.231)	[] (1.022)	4.2%	[] (372, 38%)
GQA-2-R25	[] (1.230)	[] (1.021)	4.2%	[] (371, 38%)
GQA-2-PM	[] (1.187)	[] (0.985)	0.0%	[] (373, 38%)
MQA	[] (1.271)	[] (1.055)	6.4%	[] (376, 38%)
B0-noVE	[] (1.228)	[] (1.020)	0.0%	[] (386, 39%)
GQA-2-noVE	[] (1.242)	[] (1.031)	4.2%	[] (375, 38%)

[Figure placeholder: inference_comparison_d8.pdf]

Decode tokens/s, TTFT, peak VRAM and allocated KV cache vs. prompt length, per batch size, for B0, GQA-2 and MQA.

Figure 3: Inference benchmark on one H100 (eager, BF16, greedy, 256 new tokens).

is only 5% of the bytes read per step in B0, so the ideal bandwidth-bound speedup of GQA-2 is 1.08×. At batch 64 and the full 2,048 positions, K/V is 94% of the traffic, so the ideal speedups are 1.9× (GQA-2) and 3.4× (MQA). With the window capping reads at 1,025/2,049 positions, they barely change from the 1,792- to the 8,192-token prompt, while the allocated cache grows 4× (8.25 GB for B0 at batch 64). **[Measured results (Fig. 3): HIa (decode working set vs. half the cache), HIb (batch-1 spread), HIc (batch-64 speedups vs. the ideal ratios above). Explain the gap between measured and ideal, e.g. eager kernel-launch overhead at small batch, or FA3's decode kernel not being purely bandwidth-bound.]**

5 Discussion

What the experiments establish. **[Two or three sentences: the decomposition of GQA's fixed-compute result into per-token cost, data gain and value-embedding capacity; partial RoPE; where inference gains appear. State which hypotheses held and which failed.]**

Hyp.	Prediction (registered before results)	Outcome
E1	$s \in [0.001, 0.003]$ BPB	[]
H2a	GQA-2 not resolved-worse than B0 at fixed compute	[]
H2b	GQA-2 training-throughput gain $< 5\%$	**[held (pilot: 1.6%)]**
H2c	R75 unresolved in BPB; $< 2\%$ throughput difference	[]
H3	PM within threshold of B0	[]
H4a	Monotone degradation with K/V sharing; MQA resolved-worse	[]
H4b	R25 unresolved from R75/full	[]
H5a	Removing value embeddings costs B0 a resolved loss	[]
H5b/c	GQA gap smaller without value embeddings; interaction unresolved	[]
H6	Extra tokens worth ≈ 0.005 / 0.009 BPB (GQA-2 / MQA)	[]
HIa	No K/V-sized temporary in decode	**[held in all pilots]**
HIb	Batch-1 decode within 5% across variants	[]
HIc	Batch-64 GQA speedup grows to 1,792 tokens, then plateaus	[]

Predictions vs. outcomes.

Limitations and threats to validity.

- *Statistical power.* Three seeds give s two degrees of freedom. Most variants are single runs, and no multiplicity correction is applied across ≈ 17 contrasts. A true effect of $2s$ against a three-seed mean is detected only about 40% of the time. "Unresolved" therefore means "not distinguishable at this budget", not "equal".
- *The FLOP measure.* The estimator ignores embedding lookups, norms, softmax and rotary work, so value embeddings are free and partial RoPE saves nothing by its measure. It also charges every token the full attention window rather than the causal average, which slightly understates GQA's relative saving. Measured throughput (§4.4) shows how this differs from a time budget.
- *Architectural confounds.* The value embeddings shrink with n_{kv} (E5 addresses this only at $n_{\text{kv}} \in \{4, 2\}$). The value gate becomes a single row at MQA. NorMuon's factored second moment is reduced along the longer axis, so the square MHA K/V matrices receive per-row statistics while the GQA ones receive per-column statistics. PM's wider `c_fc` gets about 6% more Muon learning rate. None of these is isolated.
- *No per-variant tuning.* All hyperparameters were tuned upstream for MHA. Part of any GQA penalty could come from this mismatch, which a learning-rate sweep at 0.5× and 2× would test.
- *Scale and duration.* All results are for a 42M-scaling-parameter model trained on about 0.45B tokens. GQA's quality cost is known to shrink with model size [1], so we make no claim about transfer to larger models.
- *Metric scope.* BPB on ClimbMix measures in-distribution language modelling only, not downstream tasks or the use of long context, which is where positional encoding choices are expected to matter most.

Future work. A d12 replication to test whether the per-token cost of GQA shrinks with size; a learning-rate sweep for B0 and GQA-2; fixing the value-embedding width to $n_{\text{head}} \cdot d_{\text{head}}$ independently of n_{kv}, which would remove the main confound; and a wall-clock-budget variant of the suite.

6 Conclusion

[Three to four sentences answering the research question with the measured numbers.]

Reproducibility. Every run is defined in one table (`experiments/config.py`), from which all configs are generated. The whole suite runs with a single Modal command, and the tables and figures in this report are regenerated without a GPU from the saved results in `results/` (see `readme.md`). Each run records its config, code commit and diff, hardware, software versions, data and tokenizer hashes, and data order.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.13245.

[2] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The Polar Express: Optimal matrix sign methods and their application to the Muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025.

[3] Federico Barbero, Alex Vitvitskyi, Christos Perivolaropoulos, Razvan Pascanu, and Petar Veličković. Round and round we go! what makes rotary positional encodings useful? In *International Conference on Learning Representations*, 2025. arXiv:2410.06205.

[4] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[5] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*, 2022. arXiv:2204.06745.

[6] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, 2021. arXiv:2103.03098.

[7] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. Nemotron-CLIMB: CLustering-based iterative data mixture bootstrapping for language model pre-training. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025. arXiv:2504.13161.

[8] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. arXiv:2010.04245.

[9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. arXiv:2203.15556.

[10] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[11] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[12] Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. https://github.com/karpathy/nanochat, 2025. Pinned commit 324e69c45d3606095adb6b409078647145165454.

[13] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. NorMuon: Making Muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025.

[14] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, 2023. arXiv:2211.05102.

[15] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[16] David R. So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V. Le. Primer: Searching for efficient transformers for language modeling. In *Advances in Neural Information Processing Systems*, volume 34, 2021. arXiv:2109.08668.

[17] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017. arXiv:1706.03762.

[19] Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. *arXiv preprint arXiv:2410.17897*, 2024.